

Assignment 5: Tsunami of Krakatau

Practical information

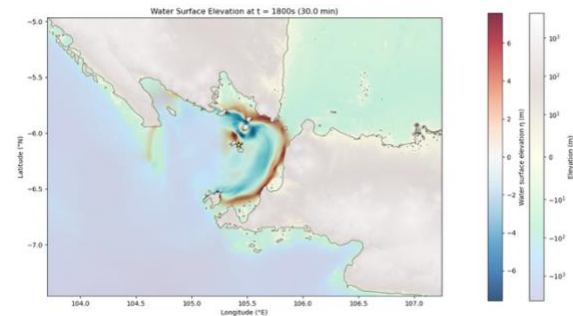
Deadline: Saturday 7/3 23.59

Resources:

- ERDA for file storage
- Nvidia GPU profiler
- ERDA DAG with Nvidia A30 vGPU instances

Handin:

- Total assignment: a report of up to 3 pages in length (excluding the code)
- You can choose between making the assignment in CUDA C++ or Python CuPy
- An up to 3-minute video where you scroll through the code and explain (see Task 1)
- Use the template on Absalon to include your codes in the report



Introduction

The Shallow Water (SW) model (see the lecture notes on Absalon; first 3 pages are enough) is the simplest numerical representation of the ocean. Still, it has reasonable skills when used to predict the evolution of storm surges or Tsunamis. Moreover, it illustrates nicely the functioning and GPU acceleration of stencil operations.

DAG

For this assignment we need `nvcc` to compile C++ and CUDA code and DAG for source code profiling and running the benchmarks. For the python version of the assignment, we need to install CuPy and Numba. You can read more about that in the file `HOW_TO_SETUP_PYTHON_ON_ERDA`.

Select a Jupyter Service

DAG MODI

Service Description

Data Analysis Gateway or DAG provides a set of interactive data analysis nodes for intermediate computation, which can be completed in a short timeframe. This means that any spawned instance is limited to 2 hours of inactivity before it will be terminated. DAG instances have access to 8 compute threads/cores and 16GB of memory

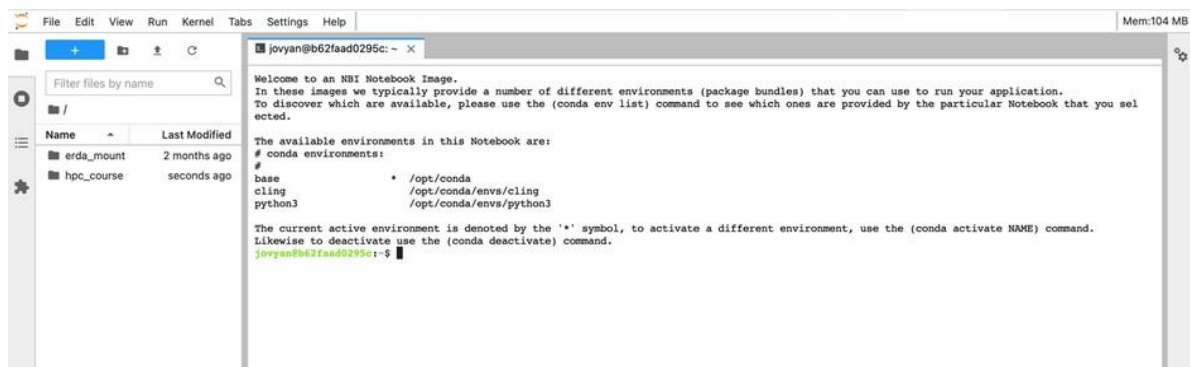
The spawned instances are non-persistent, meaning that any change made during a session is lost once the server is terminated. The only exception to this is the data that is saved in the provided mount directory (i.e. `~/work`)

For more information about how you can ease the task of configuring your instances and our future roadmap for allowing customization, check out the FAQ section "How do I install and run software XYZ in Jupyter?" at [ERDA FAQ](#)

Start DAG

Exit code: 0 Description: OK (done in 0.342s)

You can read more about DAG in the user guide: <https://erda.dk/public/ucph-erda-user-guide.pdf>
Spin up a Jupyter session on DAG selecting the “HPC GPU notebook” notebook image. In the terminal (or the folder view on the right side) you can see a number of folders.



The different folders contain:

- `erda_mount`: your own files.
- `hpc_course`: course files.

Preparations

Start by copying the exercise to your storage area and enter the folder. You can write `'ls'` to get a file listing of the folder

```
HOW_TO_SETUP_PYTHON_ON_ERDA
Makefile
bathymetry_krakatau.bin
krakatau.cpp
krakatau.cu
krakatau.py
krakatau_cupy.py
krakatau_numba.py
run_tsunami.sh
visualize.ipynb
```

For C++ and CUDA, before you can run the code, you need to compile it. This can be done by running `make` in the terminal. The `krakatau.cpp` code is very similar to `krakatau.cu` (with produced corresponding binaries `krakatau` and `krakatau_cuda`) and is there to give you a C++ comparison. The `visualize.ipynb` is for model output visualisation and analysis. The `run_krakatau.sh` script can help you with launching the code on a restricted number of streaming multiprocessors to do weak scaling.

To run the code and model 1800 seconds of Tsunami expansion on the CPU on DAG and write the model output to your storage every 120 seconds at the default resolution you can do:

```
./krakatau -scale 1.0 --tend 1800 -fout 120 --out output.dat
```

Similar, to do the same with CUDA, running on the GPU, you can do:

```
./krakatau_cuda -scale 1.0 --tend 1800 -fout 120 --out output.dat
```

For Python you run it through the terminal as a python code (REMEMBER to read `HOW_TO_SETUP_PYTHON_ON_ERDA`) with or without Numba CPU acceleration or CuPy GPU acceleration

```
python ./krakatau.py -scale 1.0 --tend 1800 -fout 120 --out output.dat
python ./krakatau_numba.py -scale 1.0 --tend 1800 -fout 120 --out output.dat
python ./krakatau_cupy.py -scale 1.0 --tend 1800 -fout 120 --out output.dat
```

The plain Python and the Numba versions are there as reference. They show how one can solve the problem either with Numpy array notation or with explicit loops. Furthermore, the Numba version has

quite good performance; similar to what is possible with a simple GPU translation. If you run the code initially without changing anything, the code will only run on the CPU; you have to move it to GPU.

You should try to visualize your output; it is quite nice to see the waves expanding on the map. Visualizing your output is also good for validation. If you would like more checks, you can add checksums. You can visualize the output by opening the notebook `visualize.ipynb`. In the output file is also stored the average code performance, measured as nano-seconds per cell update, between each snapshot. This gives you very nice performance data for benchmarking.

Code structure

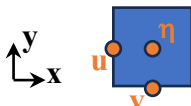
The code is relatively long, to make a realistic and scalable model, but the parts that you need to change are limited. In the code the structure is as follows (for both CUDA and Python):

- `PREC`: defines precision of all arrays. If you test on a laptop / gaming GPU make sure to use 32-bit
- `Sim_Configuration`: Read command-line arguments. *No changes needed*
- `rescale_grid`: Scales the 2D array of elevation linearly to any resolution. *No changes needed*
- `step`: implements a single timestep update. **Core routine. Update with GPU kernels**
- `ShallowWaterSolver`: set up simulation parameters and data, and use `run` to iterate over all time steps and occasionally store snapshots. **Update “run” to declare arrays for GPU and transfer of data between GPU and CPU.**
- `simulate`: init part and close down part of code, including IO. *No changes needed*
- `main`: small main routine. *No changes needed*

The `step` routine in the CUDA version has less physics and is simpler to make the workload of porting to either CUDA or CuPy similar. In addition, at the top of the CUDA code, there is a small template class `Array2D` that facilitates working with 2D arrays throughout the code. Notice that you cannot use it in the CUDA kernels but have to use one-dimensional indexing. E.g. an array accessed as `arr(j, i)` is accessed as 1D array as `arr[j*Nrow + i]`, where `Nrow` is the size of the rows (second index). One-dimensional indexing may also be necessary if you use elementwise or raw kernels in CuPy.

You have to be careful when considering the data layout. The code solves the problem on a staggered grid, where water height – η – is at the centre of a cell, while x-velocity – u – is at the x-face, and y-velocity – v – is at the y-face of a cell, the dimension of the variables are therefore

- `eta(ny, nx)`
- `u(ny, nx+1)`
- `v(ny+1, nx)`



to support the final face value on the edge of the grid. This distinction in size for the different arrays become very important when doing one-dimensional indexing of the two-dimensional arrays.

Nvidia profiler

NVIDIA profiler (nsys) enables you to understand and optimize the performance of your GPU application. Here is an example of using nsys with a CUDA GPU paralised program. In the different sections are:

[3/8]: Time spent, number of calls etc in the decorated functions. Directly related to source code ...in this example we have no decorators, so that part is skipped...

[4/8]: Timing for OS and system calls, can normally be ignored

[5/8]: Timing for CUDA specific functions

[6/8]: Timing for individual kernels

[7/8]: Memory transfer statistics from host to device and device to host

[8/8]: GPU memory usage by the program

Notice that you can also download the “report.sqlite” and “report.nsys-rep” and open it in the Nvidia Nsight Systems tool to learn more.

Going further (*optional!*): for even more profiling of the GPU, you can use “Nvidia Nsight Compute” tool, which gives a staggering amount of information. Instead of using “nsys” you need to run the “ncu” program on ERDA and then download the report to the laptop and open it in the GUI tool.

```
jovyan@09f07e790e20:~/erda_mount/Teaching/hpc_course/module5$ nsys profile --stats=true ./krakatau_cuda --scale 1 --tend 1800 --fout 60
Collecting data...
Shallow Water Solver initialized:
  Grid size: 600 x 852
  Grid spacing: dx=462.5m, dy=462.5m
.....output related to simulation.....
  Step 1340/2029, t=1188.49s, max|η|=21.0787m
Simulation complete!
```

Average performance is 0.593164 ns/grid point update

Saving results to output.dat (31 frames, 852x600 nx x ny grid)

Generating '/tmp/nsys-report-246c.qdstrm'

[1/8] [=====100%] report2.nsys-rep

[2/8] [=====100%] report2.sqlite

[3/8] Executing 'nvtx_sum' stats report

SKIPPED: /home/jovyan/erda_mount/Teaching/hpc_course_private/module5/report2.sqlite does not contain NV Tools Extension (NVTX) data.

[4/8] Executing 'osrt_sum' stats report

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
63.4	1304881708	21	62137224.2	100123524.0	2004	202327937	56798348.9	poll

.....output related to OS calls and libraries.....

[5/8] Executing 'cuda_api_sum' stats report

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
75.4	481205682	32	15037677.6	15828221.0	2154	19857141	3930785.5	cudaStreamSynchronize
19.0	121498013	9	13499779.2	105316.0	2275	120803037	40238754.7	cudaMalloc
4.1	26227493	8116	3231.6	3106.0	2895	210092	2372.8	cudaLaunchKernel
0.6	3943288	8	492911.0	615131.0	118070	827071	297709.0	cudaMemcpy
0.4	2308192	1	2308192.0	2308192.0	2308192	2308192	0.0	cudaMallocHost
0.2	1283702	9	142633.6	165688.0	4829	273018	85229.8	cudaFree
0.2	1187974	1	1187974.0	1187974.0	1187974	1187974	0.0	cudaFreeHost
0.1	536600	62	8654.8	9763.0	3627	20809	4792.4	cudaMemcpyAsync
0.0	69430	62	1119.8	932.0	431	6181	903.9	cudaEventRecord
0.0	65524	62	1056.8	862.0	461	5690	782.7	cudaStreamWaitEvent
0.0	62517	2	31258.5	31258.5	4078	58439	38439.0	cudaStreamDestroy
0.0	59761	2	29880.5	29880.5	3737	56024	36972.5	cudaStreamCreate
0.0	13245	2	6622.5	6622.5	2094	11151	6404.3	cudaEventCreateWithFlags
0.0	5850	2	2925.0	2925.0	290	5560	3726.5	cudaEventDestroy
0.0	3337	1	3337.0	3337.0	3337	3337	0.0	cuModuleGetLoadingMode

[6/8] Executing 'cuda_gpu_kern_sum' stats report

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
51.8	279391299	2029	137699.0	137633.0	133665	144288	1513.5	kernel_update_eta(double *, const double * ...
45.1	242988702	2029	119757.9	118912.0	115169	137409	3757.1	kernel_update_uv(double *, double *, ...
1.6	8576986	2029	4227.2	4160.0	3872	5696	308.5	kernel_uv_boundary(double *,double *,int,int)
1.5	8278849	2029	4080.3	4000.0	3776	5472	292.9	kernel_eta_boundary(double *, int, int)

[7/8] Executing 'cuda_gpu_mem_time_sum' stats report

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
90.3	45269758	31	1460314.8	1469093.0	1362085	1488677	31557.0	[CUDA memcpy Device-to-Host]
7.1	3563790	8	445473.8	652114.5	83776	672643	299109.2	[CUDA memcpy Host-to-Device]
2.6	1324613	31	42729.5	42432.0	41632	44768	900.7	[CUDA memcpy Device-to-Device]

[8/8] Executing 'cuda_gpu_mem_size_sum' stats report

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
126.778	31	4.090	4.090	4.090	4.090	0.000	[CUDA memcpy Device-to-Device]
126.778	31	4.090	4.090	4.090	4.090	0.000	[CUDA memcpy Device-to-Host]
22.006	8	2.751	4.092	0.511	4.096	1.854	[CUDA memcpy Host-to-Device]

Generated:

/home/jovyan/erda_mount/Teaching/hpc_course/module5/report2.nsys-rep
/home/jovyan/erda_mount/Teaching/hpc_course/module5/report2.sqlite

Task 1: CUDA / CuPy GPU parallelise the program (points 5)

The key challenge is to identify which parts of the code can reasonably be executed by the GPUs. For CUDA you have to split up the central “step” function in to a number of kernels, to avoid data races. For Python you should use CuPy instead of NumPy. Use the profiler to determine the bottlenecks. For CUDA, once the code is working, store it, and play around a bit and see if you can improve on your first try. For Python / CuPy once the code has been moved to the GPU consider fusing of operations, elementwise or raw kernels to reduce the number of GPU invocations. You have to make at least one elementwise or raw kernel.

In both cases streams may be useful for outputting data. To complete this task you should attach your code. In the report (2 points), present your overall strategy for GPU parallellising the code. Add central parts of the nsys profiler output to show how your code performs. Together with the report, submit a max 3 minute video (3 points) where you show which parts you changed, and discuss your results of the profiler.

Task 2: Weak scaling and asymptotic performance (points 5)

Measure the weak scaling of your program (2 points) using the `run_tsunami.sh` script. The script allows you to constrain the number of streaming multi-processors to an even number (from 2 to 14) using the CUDA Multi-Process Service daemon. Change “--scale” to measure the weak scaling. If you change your scale by a factor of two, you get four times the cells and therefore 8 times the workload (due to the Courant condition change the time step size correspondingly). It may therefore be useful to reduce the simulated time at higher resolution.

You should think about and explain your choice of number of grid cells and how this map to the available vGPUs. Some key figures to note: You have 14 streaming multiprocessors (compute units or SMs) available, the maximum number of threads per thread-block is 1024 and each multiprocessor can handle at most 2048 threads simultaneously. Discuss your figure (1 point).

What we often do when running on GPUs is to figure out what is the minimum size of the workload to make good use of the GPU. This can be determined by running on the full GPU while increasing the workload until the run-time increases linearly with the size of the workload. Make a plot of the asymptotic performance measured as the number of nano-seconds it takes to update a single grid-cell as a function of the number of grid cells (1 point). Based on your figure, what is a reasonable grid-size to run with on the ERDA GPU to make good use of it for Tsunami predictions (1 point)?

Bonus Task: Comparing to reality and history (no points, but fun!)

Historically, before radar or laser altimetry from space, sea height was measured in harbours or near the coast, using tide gauges (see <https://oceanservice.noaa.gov/facts/tide-gauge.html>). There exist a number of historical measurements of the Krakatau event, and I have uploaded two examples of that which gives you max height of the Tsunami at different locations and tide gauge measurements at Jakarta harbour. Can you match the time of arrival (hard!) or the amplitude of the waves in your model with historical data by changing your initial condition. How big was the initial Tsunami?